



CSC392: PRINCIPLES OF SOFTWARE ENGINEERING I

Mini Project Plan

Contents

Group Members	4
1. Introduction	5
1.1 Background	5
1.2. Aim	5
1.3. Objectives	5
1.4. Project Roles.....	5
1.5. Project Summary.....	6
2. User/Client Involvement	6
2.1. Information Provision.....	6
2.2. Resources and Facilities	6
2.3. Timing of Provision	6
3. Risks.....	7
3.1. Technical Risks	7
3.2. Schedule Risks.....	7
3.3. Resource Risks.....	7
3.4. Quality Risks	7
3.5. Operational Risks	8
4. Standards, Guidelines, and Procedures	8
4.1. Collaboration Standards	8
4.2. Coding Guidelines	8
4.2.1. General Guidelines	8
4.2.2. Backend (Node.js / Express).....	9
4.2.3. Frontend (React)	9
4.2.4. Android (Java)	9
4.2.5. Database (MySQL)	9
4.3. Documentation Procedures	10
4.4. Testing Standards	10
4.5. Quality Assurance	10

4.6. Change Management.....	10
5. Organization of the Project	10
5.1. Relation to Other Projects	10
5.2. Team Roles	11
5.3. Training Requirements	11
5.4. Training Schedule	12
6. Project Phases	12
6.1. Lifecycle Model	12
6.2. Phases and Tasks (Iterations)	12
6.3. Milestones	14
6.4. Critical Path	14
6.5. Effort Estimates (April–May)	15
7. Requirements Analysis and Design	15
7.1 Methods and Techniques	15
7.1.1. Requirements Gathering.....	15
7.1.2. Modeling Techniques.....	15
7.1.3 Validation and Refinement.....	15
7.2. Resources and Tools.....	15
7.3. Expected Deliverables	16
8. Implementation.....	16
8.1. Overview	16
8.2. Resources and Tools.....	16
8.3. Implementation Phases	17
8.4. Team Roles During Implementation	18
8.5. Cost and Estimation	18
9. Testing	18
9.1. Test Environment.....	18
9.2. Test Equipment	19
9.3. Testing Plan.....	19

9.4. Order of Integration and Testing	20
9.5. Testing Procedures	20
10. Resources	20
10.1 Overview.....	20
10.2. Resources.....	20
11. Quality Assurance.....	20
11.1. Organization	20
11.2. Procedures	21
11.2.1. Coding Standards:.....	21
11.2.2. Documentation Standards:.....	21
11.2.3. Testing Procedures:	21
11.2.4. Continuous Integration:	21
11.3. Quality Metrics	22
11.4. Review and Feedback	22
12. Changes	22
12.1. Overview.....	22
12.2. Change Management Organization	22
12.3. Procedures for Handling Changes.....	23
12.3.1. Identification	23
12.3.2. Evaluation	23
12.3.3. Approval	23
12.3.4. Implementation	23
12.3.5. Verification	23
12.3.6. Tracking.....	23
12.4. Critical Principles	23
13. Project Timeline and Dependencies (Gantt Chart Representation)	24

Group Members

Sharoon Nqobile Soulder	202304483	Software Development Leader
Dylan Senamiso Masango	202301967	Software Designer
Bandile Irvin Lokothwayo	157538	Software Programmer
Guate Banda	202301679	Requirements Analyst

1. Introduction

1.1 Background

The University of Eswatini currently relies on manual processes for exam identity verification and script collection. Students present physical exam cards and IDs, which are checked inconsistently, while exam scripts are collected at the end of sessions without a reliable tracking system. These practices create vulnerabilities such as impersonation, mismatched cards and scripts, and even loss of exam scripts.

With increasing student numbers and the need for accountability, the university requires a secure, integrated solution that links student identity to exam scripts in real time. The Universal Exam Card System is proposed to address these challenges by combining physical and digital verification methods, ensuring transparency and reducing risks.

This project builds on modern software engineering practices by integrating a node backend, a student services web extension, and a mobile app. The system will provide students with digital exam cards, invigilators with verification tools, and administrators with dashboards for monitoring exam integrity.

1.2. Aim

To develop a secure, digital examination management system that utilizes a universal identity token to automate student verification and script tracking, ensuring institutional accountability and preventing academic fraud at the University of Eswatini.

1.3. Objectives

- Develop a universal exam card that consolidates all registered modules into a single identity token for each student.
- Enable secure identity verification at exam hall entry by linking the card to the student database, preventing impersonation.
- Digitally log script collection by pairing the universal exam card with exam script scans, ensuring every script is tied to a verified student.
- Establish invigilator accountability through row-level reconciliation and a lead invigilator dashboard that tracks attendance and script collection in real time.

1.4. Project Roles

- Team Leader
- Requirements Analyst

- Software Designer
- Programmer

1.5. Project Summary

The Universal Exam Card System is a secure, cloud-enabled exam management solution developed using an iterative model to improve exam integrity and streamline student verification at the University of Eswatini. It integrates a backend API, web dashboard, and Android mobile app to provide digital exam cards with QR codes, invigilator scanning tools, and automated reporting. The system prevents fraud through identity checks and fallback mechanisms, ensures scalability with AWS deployment, and follows rigorous coding, testing, and quality assurance standards. Ultimately, it delivers a reliable, user-friendly platform that supports students, invigilators, and administrators in managing examinations efficiently

2. User/Client Involvement

The client for this project is the University of Eswatini, which will provide essential information, services, and facilities to support the development and implementation of the Universal Exam Card system.

2.1. Information Provision

- The university will supply detailed insights into current examination processes, including identity verification, exam card distribution, and script collection workflows.
- This information will be gathered through consultations with lecturers, Heads of Department (HODs), and faculty administration staff.

2.2. Resources and Facilities

- **Network Access:** School Wi-Fi will be made available to enable research, collaboration, and system testing.
- **Physical Facilities:** Classrooms and lecture halls will be used for team meetings, discussions, and project reviews.
- **Computer Laboratories:** In the event of personal device failure, the university's computer labs will serve as backup facilities for development and research activities.

2.3. Timing of Provision

- Information on exam workflows will be provided during the *requirements analysis phase*.
- Network access will be available during the *implementation and testing phases*.

- Physical facilities and computer labs will be accessible throughout the entire project lifecycle for collaboration and contingency support.

3. Risks

3.1. Technical Risks

- **Risk:** Real time exam session failure due to network outage or server downtime.
Mitigation: Implement offline functionality in the Android app with local data sync once connectivity is restored.
- **Risk:** Database errors or data loss during student identity verification.
Mitigation: Schedule regular backups, maintain transaction logs, and test with mock datasets.
- **Risk:** Integration challenges between backend, web frontend, and mobile app.
Mitigation: Use Node.js APIs as a bridge, maintain clear API documentation, and conduct early integration testing.

3.2. Schedule Risks

- **Risk:** Delays in team members uploading role documents or diagrams.
Mitigation: Set internal deadlines earlier than official submission dates and track progress using GitHub Issues.
- **Risk:** Disruptions due to student or staff protests.
Mitigation: Coordinate work remotely using online collaboration tools.

3.3. Resource Risks

- **Risk:** Limited access to licensed tools for example Visual Paradigm and IDEs.
Mitigation: Adopt free alternatives such as Draw.io, Lucidchart, and VS Code.
- **Risk:** Uneven workload distribution among team members.
Mitigation: Assign tasks based on expertise and rotate responsibilities to balance effort.

3.4. Quality Risks

- **Risk:** Inconsistent coding standards leading to errors and maintainability issues.
Mitigation: Enforce coding standards, use linting tools, and conduct peer reviews.
- **Risk:** Poor documentation hindering collaboration.

Mitigation: Use Markdown templates in the docs/ directory and require updates with each commit.

3.5. Operational Risks

- **Risk:** GitHub repository mismanagement that can result in accidental deletions, merge conflicts.
Mitigation: Adopt a branching strategy like feature branches plus pull requests, maintain backups and hold weekly training meetings on how to use the platform.
- **Risk:** Miscommunication among teammates.
Mitigation: Hold weekly check-ins, define clear role responsibilities, and use a shared progress board.

4. Standards, Guidelines, and Procedures

4.1. Collaboration Standards

- GitHub is the central collaboration platform for version control and project management.
- All development work must be committed directly to the main branch, as per lecturer requirements.
- A separate experimental repository is maintained for testing prototypes and trial features before they are finalized
- Commit messages must be clear and descriptive.

4.2. Coding Guidelines

4.2.1. General Guidelines

- Use descriptive variable and function names that clearly indicate their purpose.
- Write comments for complex logic and all API route handlers.
- Never hardcode sensitive values such as passwords, API keys, or database credentials
- All sensitive configurations must be stored in .env files and never committed to version control
- All .env files must be listed in .gitignore

4.2.2. Backend (Node.js / Express)

- Each resource must have its own route file in the routes/ directory.
- All database queries must use parameterized queries to prevent SQL injection.
- Route handlers must include error handling with try-catch.
- Return appropriate HTTP status codes for all responses (200, 400, 401, 404, 500).
- Never return raw database error objects in production, rather log them and return a generic message
- Use bcrypt for password hashing before storing credentials.

4.2.3. Frontend (React)

- API calls must use the REACT_APP_API_URL environment variable, never hardcoded URLs.
- Each page must be implemented as a separate component in the src/ directory.
- Loading states must be shown during API calls; error states must be handled gracefully.
- Error states must be handled and displayed to the user in a friendly format.
- All fetch calls must include .catch() handlers.

4.2.4. Android (Java)

- All network calls must run on background threads using OkHttp callbacks.
- UI updates after network calls must use runOnUiThread().
- The BASE_URL must be stored as a constant and updated for production deployment.
- Session data must be stored securely using SharedPreferences, never plain text files
- All activities must be registered in AndroidManifest.xml.
- Camera permissions must be requested at runtime before opening the scanner.

4.2.5. Database (MySQL)

- All tables must have a primary key.
- Foreign key constraints must be defined for relational columns.

- Column names must consistently use snake_case.
- Passwords must be stored as bcrypt hashes, never plain text.
- Sensitive operations must be logged in the scan_logs table.

4.3. Documentation Procedures

- All academic and technical documents like concept notes and role descriptions must be stored in the docs/ folder.
- Markdown (.md) format is required for readability and GitHub compatibility.

4.4. Testing Standards

- Unit tests are required for backend functions using Jest and Supertest.
- Integration tests must validate communication between React frontend, Android app, and Node.js backend.
- User Acceptance Testing (UAT) must be conducted with mock student data before deployment.
- Test results must be logged in docs/testing-report.md.

4.5. Quality Assurance

- Peer reviews are mandatory before merging code.
- Weekly check-ins will be held to review progress and identify issues early.
- Lecturer feedback must be incorporated into revisions.

4.6. Change Management

- Major changes such as design updates, database schema modifications, workflow adjustments must be documented in docs/change-log.md.
- Proposed changes must be discussed in team meetings before implementation.
- GitHub Issues will be used to track, assign, and manage change requests.

5. Organization of the Project

5.1. Relation to Other Projects

The Universal Exam Card System is grounded in the principles taught in CSC392: Practices in Software Engineering I. It is not designed as a standalone mini-project since its effectiveness

relies on integration with the university's existing systems and central database to validate student enrollments and exam schedules. By connecting directly to institutional data, the project ensures that exam verification and script tracking are seamlessly aligned with the university's established academic processes

5.2. Team Roles

- **Project Manager (Team Leader)** is responsible for guiding the overall direction of the project and ensuring smooth collaboration among team members. They oversee project planning, scheduling, and risk management, making sure milestones are achieved on time. The role involves coordinating communication, resolving conflicts, and keeping the team aligned with agreed objectives. The Team Leader also monitors progress through tools like GitHub Issues and project charts, ensuring that deliverables meet quality standards. By incorporating lecturer feedback and maintaining accountability, the Team Leader supports both the technical and organizational success of the project.
- **Requirements Analyst** confirms the team understands user needs and system goals, organizes requirements into functional and non-functional categories, documents them clearly, and supports designers and developers. Also helps verify that features match requirements to maintain quality and consistency.
- **Software Designer** translates requirements into architectural blueprints and design models. They create UML diagrams such as ERD, Sequence, and Activity diagrams to define system structure, behavior, and data flow. Ensure the system is scalable, maintainable, and aligned with requirements, providing a clear framework that guides consistent development across backend, frontend, and mobile components by applying design principles and patterns.
- **Programmer/Developer** is responsible for implementing system functionality across backend, frontend, and mobile components. They design and write code, integrate modules, and ensure proper interaction with the database. In addition to development, the Programmer prepares and executes test cases, runs unit and integration tests, and verifies that features meet documented requirements. They also document test results in docs/testing-report.md, helping maintain quality and consistency in the absence of a dedicated tester.

5.3. Training Requirements

- **GitHub Workflow:** Master version control, including branching, pull requests, and commit standards across web, backend, and Android repositories.

- **Diagramming & Planning Tools:** Be proficient in Visual Paradigm, Lucidchart, or Draw.io to create and interpret UML diagrams, Gantt charts, and PERT diagrams for critical path analysis.
- **Coding & Security Standards:** Familiarization with project guidelines such as JWT authentication, parameterized SQL queries, and Android background thread management.
- **Project Management Tools:** Understanding scheduling techniques and dependency tracking using Gantt or PERT charts.

5.4. Training Schedule

- **Week 1:** GitHub basics, repository initialization, and AWS CLI setup.
- **Week 2:** UML diagramming workshop and MySQL Workbench schema design training.
- **Week 3:** Coding standards orientation (Node.js/React/Java) and security protocol briefing.
- **Week 4:** Testing methodology training, focusing on Jest, Supertest, and Android Emulator usage

6. Project Phases

6.1. Lifecycle Model

We adopted the **Iterative Model** because the project has already progressed outside the traditional sequence of the System Development Life Cycle. A workflow and prototype were created before requirements were fully defined, making a strictly sequential approach unsuitable. The Iterative Model allows us to refine requirements, design, and implementation in cycles, incorporating feedback at each stage. This ensures flexibility, continuous improvement, and proper documentation while meeting fixed academic deadlines

6.2. Phases and Tasks (Iterations)

- **Iteration 1 (10–14 April)**
 - Proposal PDF which consisted of problem statement, solution, workflow and benefits created and Project approved.
 - Initial prototype sketches shared in team discussions.
 - Unofficial GitHub repository initialized with first commit.

- *Milestone: Concept note and initial prototype established.*
- **Iteration 2 (14–24 April)**
 - Consecutive commits refining dashboard, login routes, and exam card handling on unofficial prototype GitHub repository.
 - Official GitHub repository created (Task 2 submission).
 - Problem statement rewritten; aims and objectives documented.
 - *Milestone: Official repository and academic documentation completed.*
- **Iteration 3 (24–25 April)**
 - Project plan drafted, including lifecycle model, phases, milestones, critical path, and effort estimates.
 - *Milestone: Project plan uploaded to documentation.*
- **Iteration 4 (26 April – 3 May)**
 - Requirements Analyst: Begin gathering and documenting functional and non-functional requirements; draft use cases and diagrams.
 - Designer: Continue UX/UI prototype contributions; start workflow and UML model creation.
 - Programmer: Extend prototype functionality; run initial unit tests; document results in docs/testing-report.md.
 - *Milestone: Draft requirements and design models completed.*
- **Iteration 5 (4–10 May)**
 - Refine requirements and design models based on feedback.
 - Programmer integrates backend logic with database schema; frontend and mobile components expanded.
 - Designer improves UX/UI and aligns models with requirements.
 - *Milestone: Prototype refined with functional requirements integrated.*
- **Iteration 6 (11–17 May)**
 - Programmer continues coding and testing in parallel (backend, web, mobile).

- Requirements Analyst finalizes documentation of requirements.
 - Designer completes UML and workflow models.
 - Team Leader approves environment setup for AWS server.
 - *Milestone: Requirements finalized, design package completed, AWS environment prepared.*
- **Iteration 7 (18–27 May)**
 - Programmer runs integration and acceptance tests with mock student data.
 - Designer polishes UX/UI and ensures consistency with models.
 - Requirements Analyst validates system against documented requirements.
 - Team Leader coordinates deployment package and final submission.
 - Milestone: Final submission delivered (27 May).

6.3. Milestones

- Proposal and concept note completed (10 April).
- Initial prototype and GitHub setup (14 April).
- Official repository, rewritten problem statement, aims and objectives (24 April).
- Project plan drafted (25 April).
- Requirements and design models drafted (3 May).
- Prototype refined with requirements integrated (10 May).
- Requirements finalized, design package completed, AWS environment prepared (17 May).
- Final submission delivered (27 May).

6.4. Critical Path

- Requirements → Design → Implementation → Testing → Deployment.
- Each iteration builds on the previous one; delays in requirements or design directly affect later cycles.
- Parallel tasks reduce time but depend on requirements and design completion.

6.5. Effort Estimates (April–May)

- Requirements: ~20 person-hours.
- Design: ~25 person-hours.
- Implementation: ~40 person-hours.
- Testing: ~20 person-hours.
- Deployment: ~15 person-hours.

Total estimated effort: ~120 person-hours across 7 iterations (10 April – 27 May).

7. Requirements Analysis and Design

7.1 Methods and Techniques

7.1.1. Requirements Gathering

- Collect functional and non-functional requirements through team discussions, observation of exam workflows, and stakeholder input.
- Document requirements in a structured specification format.

7.1.2. Modeling Techniques

- Use Case Diagrams to capture interactions between students, invigilators, administrators, and the system.
- Context Diagram and Data Flow Diagrams (DFD) to show information flow and system boundaries.
- Entity-Relationship Diagram (ERD) to define database schema and relationships.
- Sequence and Activity Diagrams to model system behavior and process flows.

7.1.3 Validation and Refinement

- Iterative reviews of requirements and models with the team.
- Feedback loops to adjust requirements and design before implementation.

7.2. Resources and Tools

- **Diagramming Tools:** Visual Paradigm, Lucidchart, or Draw.io for UML diagrams, workflow models and Gantt charts or PERT diagrams

- **Documentation Tools:** Markdown files in GitHub for example docs/requirements.md and docs/design.md for collaborative editing and version control.
- **Project Management Tools:** GitHub Projects or Trello for task tracking; Gantt or PERT charts for scheduling and critical path analysis.
- **Collaboration Platform:** GitHub repository for storing requirements, design models, and linking commits to documentation.

7.3. Expected Deliverables

- Requirements Specification Document consisting of functional, non-functional, constraints.
- Approved Use Case, Context, and DFD diagrams.
- ERD, Sequence, and Activity diagrams.
- Gantt chart
- Design package uploaded to docs/ directory.

8. Implementation

8.1. Overview

The implementation phase focuses on building, integrating, and deploying the Universal Exam Card System across its three main components; backend, web frontend, and mobile app. This phase uses the iterative model, allowing continuous testing, prototyping, and refinement while maintaining alignment with requirements and design outputs.

8.2. Resources and Tools

Resource	Specification
Development PC	Minimum 8 GB RAM, 256 GB SSD, Windows 10/11 or macOS
Android Device	Android 7.0 (API 24) or higher with camera for QR scanning
Network	Stable internet connection for cloud deployment and API calls
AWS Account	Free tier account sufficient for development and small-scale production

Table 1. Hardware Requirements

Software	Version / Notes
Node.js	v18 or higher
MySQL Server	v8.0 or higher
Android Studio	Hedgehog or later with Gradle 9.3.1
React	v18 with Create React App
Java JDK	Version 11 for Android development
AWS Services	RDS, Elastic Beanstalk, S3, Amplify, EC2

Table 2. Software Requirements

Tool	Component	Purpose
Visual Studio Code	React Frontend + Node.js Backend	Primary code editor with JS/Node extensions
Android Studio	Android App	Official IDE for Java Android development
MySQL Workbench	Database	Schema design, query execution, and data management
Git + GitHub	All components	Version control and collaboration
Postman	Backend + API	Endpoint testing and documentation
Figma	UI/UX	Design mockups before implementation
AWS CLI	Cloud	Deploy and manage AWS services

Table 3. Implementation: Development tools

8.3. Implementation Phases

Phase	Milestone	Description
1	Database Setup	Design and implement MySQL schema, insert test data, verify relationships.
2	Backend API	Implement Express routes, connect to MySQL, test endpoints with Postman.
3	React Dashboard	Build student login, dashboard, PDF report, and QR exam card.
4	Android App	Develop invigilator login, home screen, QR scanner, and manual entry.

Phase	Milestone	Description
5	Integration Testing	Test all components end-to-end with mock student data.
6	Cloud Deployment	Deploy database to AWS RDS, backend to Elastic Beanstalk, frontend to Amplify.
7	User Acceptance Testing (UAT)	Conduct UAT with real invigilators and mock student data.
8	Production Launch	Publish Android app and go live with all cloud services.

Table 4. Implementation Phases

8.4. Team Roles During Implementation

- **Requirements Analyst:** Continues refining requirements and validating system behavior.
- **Designer:** Enhances UX/UI and ensures design consistency across platforms.
- **Programmer:** Develops, tests, and deploys backend, web, and mobile components.
- **Team Leader:** Oversees environment setup (AWS), approves milestones, and coordinates integration.

8.5. Cost and Estimation

Resource	Estimated Cost	Notes
AWS Free Tier (12 months)	\$0	EC2, Amplify within limits.
Total Effort	~120 person-hours	Distributed across design, coding, testing, and deployment.

Table 5. Implementation: Cost and Estimation

9. Testing

9.1. Test Environment

Testing will be conducted across all four system components, both local and cloud environments to ensure reliability and scalability.

Component	Environment	Description
Backend	Local (Node.js v18 + MySQL v8.0) and AWS Elastic Beanstalk	API routes tested locally and deployed to cloud for integration.
Web Frontend	Local React development server and AWS Amplify	Browser-based testing on Chrome, Firefox and Edge.
Mobile App	Android Studio emulator and physical Android devices (API 24+)	Tests, QR scanning, login, and venue verification.
Database	MySQL Workbench and AWS RDS	Schema validation, data integrity checks, and mock student data insertion.

Table 6. Test Environments for System Components

9.2. Test Equipment

- Development laptops with VS Code and Android Studio.
- Android devices for real-world QR scanning tests.
- Stable internet connection for cloud deployment and API calls.
- AWS RDS and Elastic Beanstalk instances for live integration tests.
- Postman for API testing and Jest + Supertest for automated backend tests.

9.3. Testing Plan

Testing follows an iterative approach, ensuring each component is validated before integration.

- **Unit Testing**
 - Backend functions and route handlers tested with Jest and Supertest.
 - React components tested with Jest and React Testing Library.
 - Android activities tested using JUnit and instrumentation tests.
- **Integration Testing**
 - Verify communication between backend APIs and web/mobile clients.
 - Ensure data consistency across database and UI components.
- **System Testing**

- End-to-end workflow: student card scan → verification → script logging → dashboard update.
- Test concurrency for multiple students entering simultaneously.
- **User Acceptance Testing (UAT)**
 - Conducted with mock student data and invigilator accounts.
 - Validate real-world scenarios such as wrong venue, duplicate scan, and manual entry.

9.4. Order of Integration and Testing

1. Test backend modules independently.
2. Integrate backend with web frontend → run integration tests.
3. Integrate backend with mobile app → run integration tests.
4. Combine all components → perform system testing.
5. Conduct UAT before deployment to AWS.

9.5. Testing Procedures

- Define test cases in docs/testing-report.md.
- Record expected vs. actual results for each test case.
- Document bugs and issues in GitHub Issues for tracking.
- Retest after fixes to confirm resolution.
- Generate final test report summarizing coverage and outcomes.

10. Resources

11. Quality Assurance

11.1. Organization

- **Team Leader:** Oversees quality assurance activities, ensures standards are followed, and coordinates reviews.

- **Requirements Analyst:** Validates that the system meets documented functional and non-functional requirements.
- **Designer:** Reviews UX/UI consistency and ensures design models align with requirements.
- **Programmer:** Implements coding standards, runs automated tests, and documents results.
- **Peer Review:** All code and documentation must be reviewed by at least one teammate before merging into the main branch.

11.2. Procedures

11.2.1. Coding Standards:

- Backend follows Node.js/Express guidelines (parameterized queries, error handling, bcrypt hashing).
- Frontend follows React conventions (component separation, error handling, environment variables).
- Android app follows Java/Android best practices (background threads, runtime permissions, secure storage).
- Database follows schema rules (primary keys, foreign keys, snake_case, bcrypt password storage).

11.2.2. Documentation Standards:

- All documents stored in docs/ folder in Markdown format.
- Updates must accompany commits to keep repository synchronized.

11.2.3. Testing Procedures:

- Unit tests for backend functions and route handlers.
- Integration tests for web and mobile components.
- System testing of end-to-end workflows.
- User acceptance testing with mock student data.

11.2.4. Continuous Integration:

- GitHub Actions runs automated tests on every commit.
- Builds must pass before merging into the main branch.

11.3. Quality Metrics

- **Reliability:** No critical bugs in final submission.
- **Usability:** Clear user interface for students, invigilators, and administrators.
- **Performance:** System responds to card scans and script logging within 2 seconds.
- **Maintainability:** Code is modular, documented, and reusable.
- **Security:** JWT authentication, HTTPS enforcement, and AWS security groups applied.

11.4. Review and Feedback

- Weekly team check-ins to review progress and catch errors early.
- Lecturer feedback incorporated into revisions.
- Issues tracked in GitHub for transparency and accountability.

12. Changes

12.1. Overview

Changes during the software development process are inevitable due to evolving requirements, design refinements, and testing feedback. To ensure these changes are managed systematically, the team will follow a structured **change management procedure**. A more detailed version may be documented separately in a **Configuration Management Plan**.

12.2. Change Management Organization

- **Team Leader:** Approves or rejects major changes, ensures alignment with project goals.
- **Requirements Analyst:** Documents changes to functional and non-functional requirements.
- **Designer:** Updates design models and UX/UI mockups to reflect approved changes.
- **Programmer:** Implements code changes, ensures backward compatibility, and updates tests.
- **All Team Members:** Record changes in GitHub Issues and update documentation accordingly.

12.3. Procedures for Handling Changes

12.3.1. Identification

- Any team member who discovers a need for change (bug fix, new requirement, design adjustment) must log it in GitHub Issues.
- Each change request must include a description, rationale, and potential impact.

12.3.2. Evaluation

- The team leader and analyst assess the impact on scope, schedule, and resources.
- Changes are categorized as minor (UI tweaks, bug fixes) or major (new features, database schema updates).

12.3.3. Approval

- Minor changes may be approved by the team leader directly.
- Major changes require team discussion and consensus before implementation.

12.3.4. Implementation

- Programmer applies changes in a separate branch.
- Designer updates models and mockups if applicable.
- Documentation in docs/change-log.md is updated to reflect the change.

12.3.5. Verification

- Changes undergo unit and integration testing.
- Results are logged in docs/testing-report.md.

12.3.6. Tracking

- All changes are tracked in GitHub Projects (Kanban board: To Do → In Progress → Done).
- Weekly reviews ensure no undocumented changes are introduced.

12.4. Critical Principles

- **Transparency:** Every change must be documented and visible to the team.
- **Traceability:** Each change links back to its origin (requirement, bug report, feedback).

- **Accountability:** Only approved changes are merged into the main branch.
- **Consistency:** Documentation, code, and design models must remain synchronized.

13. Project Timeline and Dependencies (Gantt Chart Representation)

Iteration	Dates	Tasks	Milestones	Dependencies
1	10–14 April	Project approval; proposal PDF created; initial prototype sketches; GitHub repo initialized	Concept note and initial prototype established	None (start)
2	14–24 April	Consecutive commits refining dashboard, login routes, exam card handling; official repo created; problem statement rewritten; aims & objectives documented	Official repository and academic documentation completed	Iteration 1
3	24–25 April	Draft project plan (lifecycle model, phases, milestones, critical path, effort estimates)	Project plan uploaded to docs/	Iteration 2
4	26 Apr – 3 May	Requirements Analyst drafts functional & non-functional requirements, use cases, diagrams; Designer continues UX/UI prototype and workflow models; Programmer extends	Draft requirements and design models completed	Iteration 3

		prototype functionality and runs initial unit tests		
5	4–10 May	Refine requirements and design models; Programmer integrates backend logic with database schema; frontend and mobile expanded; Designer improves UX/UI	Prototype refined with functional requirements integrated	Iteration 4
6	11–17 May	Programmer continues coding and testing; Requirements Analyst finalizes documentation; Designer completes UML and workflow models; Team Leader approves AWS environment setup	Requirements finalized, design package completed, AWS environment prepared	Iteration 5
7	18–27 May	Programmer runs integration and acceptance tests; Designer polishes UX/UI; Requirements Analyst validates system against requirements; Team Leader coordinates deployment package	Final submission delivered (27 May)	Iteration 6

Table 11. Gantt Chart